JALA UNIVERSITY

Web Development

Laboratory 7: Pulimiento del Proyecto Final, Documentación y Ensayo de Presentación

ESTUDIANTE

Catriel Pereira Torrez

INSTITUCIÓN

Jala University

MATERIA

Web Development (CSWB-366)

PROYECTO CAPSTONE

University Library SPA

FECHA

20 de octubre, 2026

1. Resumen Ejecutivo y Entregables

Este laboratorio documenta la fase culminante del proyecto Capstone: la transición desde una aplicación con funcionalidades completas hacia un producto de software de grado de producción, altamente pulido, accesible, adaptado con enfoque **Mobile-First**, respaldado por una infraestructura en la nube reproducible mediante Infraestructura como Código (IaC) en AWS, y documentado para la presentación final.

Entregables del Laboratorio

Entregable	Enlace / Ubicación	Descripción
Aplicación Capstone en Vivo	https://d1376jrski7-xuw.cloudfront.net	Desplegada globalmente mediante Amazon CloudFront CDN con HTTPS y compresión Brotli/Gzip.
Backend API Gateway & Lambda	https://b9fknpj872.execute-api.us-east-1.amazonaws.com	API REST serverless en Node.js 20.x conectada a PostgreSQL en Amazon RDS.
Repositorio de Código Fuente	GitHub: Catriel42/university-library	Repositorio completo con `README.md` profesional, código fuente y configuración de Terraform.
Repositorio Institucional	GitLab Jala University	Espejo sincronizado de entrega académica.
Borrador de Diapositivas	`docs/labs/presentation-slides-draft.md`	Guión estructurado y diapositivas para la defensa y demo en vivo.

2. Actividad #1: Pulido Final y Corrección de Errores (UI/UX Mobile-First)

2.1 Diagnóstico de Problemas en Experiencia Móvil

Durante la auditoría exhaustiva de UI/UX previa a la entrega final, se identificaron deficiencias críticas de adaptabilidad en dispositivos móviles (< 768px):

1. **Navegación Inaccesible en Dispositivos Móviles:**
   En la barra de navegación superior, los enlaces directos a *"My Loans"*, *"Reservations"* y *"My Books"* estaban marcados con `hidden md:flex`. En teléfonos móviles, un usuario autenticado no tenía forma visual de acceder a sus préstamos ni a sus reservas desde el encabezado.
2. **Cuadrícula Rígida de Resultados del Catálogo:**
   El componente `CatalogResultsGrid` utilizaba `grid-cols-2` en todos los anchos de pantalla pequeños. En pantallas de 360px a 400px (smartphones estándar), las tarjetas de libros quedaban comprimidas a menos de 160px de ancho, provocando truncamiento severo de títulos y desalineación de los indicadores de copias disponibles.
3. **Formularios y Controles con Touch Targets Pequeños:**
   La barra de búsqueda y los selectores de categorías tenían botones compactos que dificultaban el toque en pantallas táctiles, incumpliendo la directriz WCAG 2.1 de tamaño de objetivo mínimo (44×44 px44 \times 44\text{ px}).
4. **Desbordamiento Horizontal en Historial:**
   La tabla de historial de préstamos ubicaba la fecha de devolución en una columna fija a la derecha, comprimiendo los títulos largos en smartphones.
5. **Selectores de Pestañas Angostos:**
   En *"My Loans"* y *"My Books"*, los botones de pestañas tenían un ancho estático `max-w-xs`, luciendo desalineados en pantallas móviles completas.

2.2 Refactorizaciones y Mejoras Mobile-First Implementadas

Para solucionar estos problemas de forma integral, se aplicaron las siguientes optimizaciones en el código del frontend:

A. Barra de Navegación con Menú Hamburguesa y Drawer Deslizable (`AppLayout.tsx`)

- Se implementó un estado reactivo `isMobileMenuOpen` controlado por un botón táctil con accesibilidad completa (`aria-label`, `aria-expanded`) utilizando los iconos `Menu` y `X` de Lucide.
- En pantallas móviles (< 768px), el botón activa un drawer deslizable con efecto *backdrop-blur* (`bg-bg/95 backdrop-blur-xl`) que presenta:
  - Ficha de perfil del estudiante autenticado con nombre, correo y avatar.

- Enlaces de navegación con iconos y resaltado de ruta activa (`Catalog`, `My Loans`, `Reservations`, `My Books`).
- Botón táctil ergonómico de cierre de sesión (`Sign Out`) o llamada a la acción para iniciar sesión (`Sign In`).
- Cierre automático al interactuar con cualquier enlace o cambiar de vista.

B. Cuadrícula Adaptativa del Catálogo (`CatalogResultsGrid.tsx`)

- Se rediseñó el layout de cuadrícula mediante clases de Tailwind responsive:

```
className="grid w-full grid-cols-1 gap-4 min-[440px]:grid-cols-2 md:grid-cols-3 md:gap-6 xl:grid-cols-4"
```

- En pantallas ultra-estrechas (< 440px), la vista se presenta en 1 columna cómoda con portadas legibles y botones de acción accesibles. A partir de 440px se expande a 2 columnas, 3 en tablets y 4 en pantallas de escritorio.

C. Touch Targets Ergonómicos y Barra de Búsqueda Flexible (`CatalogSearchForm.tsx` & `CategoryFilterBar.tsx`)

- Los selectores de alcance de búsqueda (`All | Title | Author`) ahora ocupan el ancho completo en móviles (`w-full sm:w-auto`).
- El input de búsqueda incluye un padding derecho adecuado (`pr-24`) para evitar que el botón "Search" y el botón "X" de borrado se superpongan con el texto ingresado.
- La barra de categorías de libros cuenta con desplazamiento horizontal fluido con inercia táctil (`scrollbar-none overflow-x-auto`).

D. Historial de Préstamos Adaptable (`LoanHistoryTable.tsx`)

- Se rediseñaron las filas para apilarse en móvil:

```
className="flex flex-col justify-between gap-3 rounded-xl border p-4 sm:flex-row sm:items-center sm:gap-4"
```

  La insignia *"Returned [date]"* se reubica debajo de la información del libro en smartphones, aprovechando todo el ancho disponible sin apretar los textos.

E. Selectores de Pestañas de Ancho Completo (`MyLoansPage.tsx` & `MyBooksPage.tsx`)

- Se actualizaron los contenedores de pestañas a `w-full sm:max-w-xs`, permitiendo una distribución táctil simétrica al 50% de la pantalla en dispositivos móviles.

2.3 Corrección de Bug Crítico: Sesión y Cookies en Producción (CloudFront Reverse Proxy)

El Problema

Al desplegar inicialmente el frontend en S3/CloudFront (`d1376jrski7xuw.cloudfront.net`) y el backend en API Gateway (`b9fknpj872.execute-api.us-east-1.amazonaws.com`), el navegador detectaba las peticiones AJAX como **Cross-Site**. Debido a la política `SameSite=Lax` configurada en la cookie `token`, los navegadores modernos (Chrome, Safari, Firefox) **bloqueaban el envío de la cookie en peticiones subsiguientes**:

- Al intentar pedir prestado un libro (`POST /api/loans`), el servidor respondía con error `401 Unauthorized` ("No estás autenticado").
- Al recargar la página (*refresh*), la llamada a `/api/auth/me` no enviaba la cookie y la sesión se perdía.

La Solución Arquitectónica

En lugar de forzar cookies de terceros (bloqueadas por las políticas ITP de Apple y Privacy Sandbox de Google), se reconfiguró la distribución de **Amazon CloudFront** en `infra/s3_cloudfront.tf` para actuar como **Reverse Proxy Same-Origin**:

1. Se añadió un segundo origen en CloudFront apuntando al **Amazon API Gateway HTTP API**.
2. Se crearon dos comportamientos de caché priorizados (*Ordered Cache Behaviors*) para `/api/*` y `/health` con la política gestionada `Managed-AllViewerExceptHostHeader`.
3. El frontend se configuró para realizar peticiones relativas a `/api`.
4. **Resultado:** Tanto la aplicación React como la API REST corren bajo el mismo dominio (`https://d1376jrski7xuw.cloudfront.net`). La cookie de autenticación se gestiona como **1st-party cookie**, persistiendo la sesión de forma transparente y sin generar peticiones preflight CORS innecesarias.

2.4 Verificación de Calidad y Suite de Pruebas

Se ejecutó la suite completa de pruebas unitarias, de integración y de componentes en el monorepo:

- **Frontend:** 47 suites de prueba aprobadas, 130 pruebas unitarias y de componentes aprobadas (incluyendo la nueva prueba de interacción del menú hamburguesa móvil en `AppLayout.test.tsx`).
- **Backend:** 10 suites de prueba aprobadas, 31 pruebas de reglas de negocio y concurrencia aprobadas.
- **Verificación de Tipos TypeScript (`pnpm typecheck`):** 0 errores en `@university-library/shared`, `@university-library/backend` y `@university-library/frontend`.
- **Total: 161 pruebas automáticas aprobadas (57 suites de prueba).**

3. Actividad #2: Documentación del Proyecto y Repositorio

Se actualizó exhaustivamente el archivo `README.md` en la raíz del repositorio, asegurando el cumplimiento de todos los estándares de documentación profesional:

1. **Insignias Dinámicas (*Badges*):** Estado de despliegue en CloudFront, API Gateway, Amazon RDS y conteo de pruebas pasando.
2. **Enlaces en Vivo:** Direcciones de acceso público verificadas tanto para la interfaz de usuario como para la API de salud y catálogo.
3. **Descripción y Reglas de Negocio:** Síntesis de las reglas de préstamo (BR-01 a BR-09), cola de reserva FIFO (BR-10) y ventana exclusiva de recogida de 48 horas (BR-11 a BR-12).
4. **Guía de Puesta en Marcha Local (*Getting Started*):** Instrucciones claras de clonado, instalación con `pnpm`, variables de entorno para desarrollo y comandos de ejecución.
5. **Documentación de Pruebas e Infraestructura:** Comandos para ejecutar Jest, TypeScript typecheck y Terraform.
6. **Mapeo del Repositorio de Documentación:** Vínculos a la especificación SRS (IEEE 29148), diseño de base de datos, registros de decisiones de arquitectura (ADRs) y plan de desarrollo.

4. Actividad #3: Arquitectura y Despliegue en la Nube (AWS + Terraform)

4.1 Infraestructura como Código (IaC)

Toda la infraestructura cloud fue aprovisionada de forma reproducible utilizando **Terraform** en el directorio `infra/`:

- **`infra/versions.tf`**: Configuración de proveedores AWS (≥5.40\ge 5.40) y Random.
- **`infra/network.tf`**: Subnet group de base de datos y Security Group con regla de entrada en puerto 5432 con cifrado SSL obligatorio.
- **`infra/rds.tf`**: Instancia gestionada de PostgreSQL 16.3 (`db.t4g.micro`, 20 GB gp3, Single-AZ, sin auto-escalado de almacenamiento para garantizar **costo cero** dentro del Free Tier de AWS).
- **`infra/lambda_api.tf`**: Función AWS Lambda en Node.js 20.x empaquetada con `esbuild` y dependencias de Prisma (29 MB), expuesta mediante Amazon API Gateway HTTP API v2.
- **`infra/s3_cloudfront.tf`**: Bucket Amazon S3 privado con Origin Access Control (OAC) y distribución CloudFront CDN con enrutamiento de errores SPA (403/404 →\to `/index.html` 200) y reverse proxy hacia la API.
- **`infra/outputs.tf`**: Exportación de endpoints, buckets y dominios aprovisionados.

4.2 Diagrama de Despliegue (Mermaid)

Se diseñó y modeló el diagrama de arquitectura y despliegue cloud en **Mermaid**, disponible en alta resolución en el repositorio:

- **Archivo fuente Mermaid:** `docs/architecture/diagrams/deployment-aws.mmd`
- **Render vectorial SVG:** `docs/architecture/diagrams/AWS_Deployment_Architecture_Mermaid.svg`
- **Render PNG:** `docs/architecture/diagrams/AWS_Deployment_Architecture_Mermaid.png`

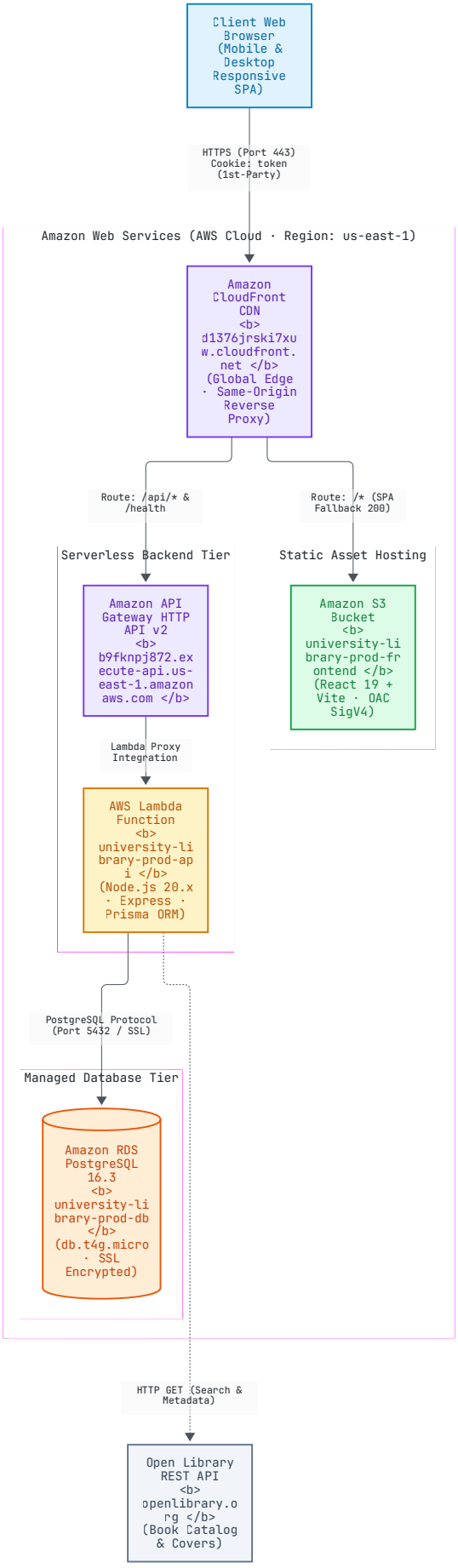


Figura 1 — Diagrama de Despliegue (Mermaid)

5. Actividad #4: Preparación de la Presentación Final (Diapositivas y Guión)

Se preparó un borrador completo de 10 diapositivas para la defensa del Capstone, disponible en `docs/labs/presentation-slides-draft.md`.

5.1 Estructura de las Diapositivas

1. **Portada:** Título del proyecto, autor, institución, fecha y enlaces en vivo.
2. **El Problema & Contexto:** Desafíos de gestión bibliográfica universitaria, sincronización de copias físicas y catálogos públicos externos.
3. **La Solución (University Library SPA):** Plataforma web moderna, en tiempo real, con diseño Mobile-First y catálogo Open Library.
4. **Arquitectura del Sistema (Cloud Native):** Diagrama C4 y desglose de servicios en AWS (CloudFront, S3, API Gateway, Lambda, RDS).
5. **Reglas de Negocio Clave (Lending & Reservations):** Concurrencia segura, límite de 5 préstamos, renovaciones restringidas (+7 días) y cola FIFO de reservas con ventana de 48 horas.
6. **Experiencia de Usuario & Mobile-First:** Menú interactivo, tarjetas adaptativas, accesibilidad WCAG 2.1 AA y modo oscuro/claro.
7. **Calidad de Software & Testing:** Estrategia con Jest y React Testing Library (161 pruebas pasando, 0 errores de tipos en TypeScript).
8. **Infraestructura como Código (IaC):** Automatización con Terraform y optimización de costos en AWS Free Tier.
9. **Demostración en Vivo (Demo Script):** Flujo de trabajo en tiempo real paso a paso.
10. **Conclusiones & Lecciones Aprendidas:** Logros del proyecto, retos técnicos superados y trabajo futuro.

5.2 Guión Ensayado para la Demostración en Vivo (5-7 Minutos)

Tiempo	Sección	Acción en Vivo	Puntos Clave a Explicar
0:00 - 1:00	Introducción	Abrir la app en https://d1376jrski7-xuw.cloudfront.net	Presentar el propósito del proyecto, destacar el despliegue global en CloudFront y el tema visual oscuro/claro.
1:00 - 2:00	Catálogo y Búsqueda	Buscar un autor (ej. *"Tolkien"*), aplicar filtro de materia (*"Fiction"*), ordenar por popularidad	Demostrar integración con la API de Open Library combinada en tiempo real con la disponibilidad de copias físicas de PostgreSQL.

Tiempo	Sección	Acción en Vivo	Puntos Clave a Explicar
2:00 - 3:00	Vista Mobile-First	Activar DevTools en modo iPhone/Android (390px)	Mostrar la barra de navegación con menú hamburguesa, el drawer deslizable accesible, y la cuadrícula adaptable de libros sin desbordamiento.
3:00 - 4:15	Préstamo y Reglas	Iniciar sesión, pedir prestado un libro disponible, verificar en *"My Loans"*, intentar renovar	Explicar la transacción serializable en base de datos, el cálculo automático de fecha de vencimiento a 14 días y la regla de máximo 2 renovaciones (+7d).
4:15 - 5:15	Cola de Reserva FIFO	Buscar un libro con 0 copias disponibles, pulsar *"Place Hold"*, abrir *"Reservations"*	Mostrar la posición en la cola FIFO (#1). Explicar cómo al devolver la copia se activa la ventana exclusiva de 48 horas para recogida.
5:15 - 6:00	Reading Tracker & Devolución	Marcar un libro como *"Reading"*, moverlo a *"Completed"* en *"My Books"*, devolver el préstamo	Demostrar persistencia en el log de lectura y registro en el historial permanente de préstamos.
6:00 - 6:30	Conclusión y QA	Mostrar la suite de 161 pruebas pasando en terminal	Resumir el cumplimiento de los requerimientos SRS, la robustez de la arquitectura serverless y dar paso a preguntas.

6. Conclusiones y Estado Final

El proyecto Capstone **University Library SPA** ha completado exitosamente su ciclo de desarrollo, pruebas, optimización mobile-first y despliegue en la nube:

1. **Funcionalidades:** 100% de los requerimientos funcionales (FR) y reglas de negocio (BR) implementados y probados.
2. **Experiencia de Usuario:** Interfaz responsiva con enfoque Mobile-First, probada en múltiples factores de forma y con navegación accesible.
3. **Estabilidad y Seguridad:** Autenticación con cookies Same-Origin vía CloudFront Reverse Proxy, TLS obligatorio en base de datos y contraseñas hasheadas con bcrypt.
4. **Infraestructura:** Desplegada en AWS mediante Terraform bajo parámetros estrictos de costo cero ($0.00) en el Free Tier.
5. **Listo para Defensa:** Documentación completa y guión de presentación estructurado para la evaluación final.